SpriteCraft: Texture Style Transfer for Minecraft Resource Packs

Ray Zhang · Shanghai Jiao Tong University

上海交通大学 SHANGHAI JIAO TONG UNIVERSITY

8.7M Dual diffusion network architecture

11x Resource packs validated

Consistent results for multiple 0-1 shot textures

Ensembled models to provide precision for all

1. Motivation

Minecraft resource packs provide alternative textures for the gameplay, but they are often incomplete regarding new updates as well as modded contents.

Filling in these missing textures is difficult work, with traditional diffusion failing to grasp the low-res nature of pixel art, and Transfusion-based models computationally expensive.

Our Goal

Provide a lightweight, diffusion-based framework for training models to predict textures for resource packs.

2. Building Process

The progress of the project is far from smooth. Here we demonstrate the route which we took and lessons learnt.

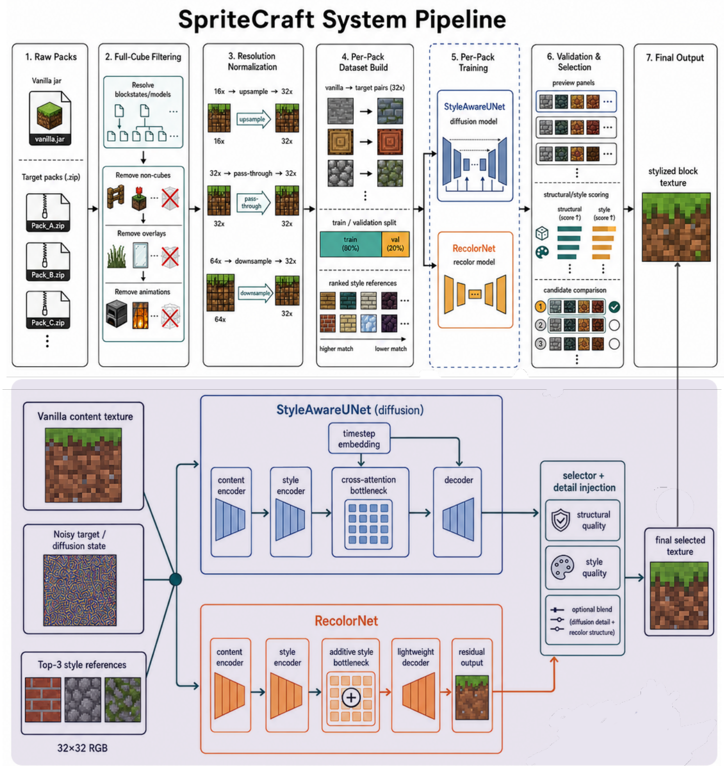
Continuous vs. Discrete Diffusion. Originally I started with discrete diffusion, tokenizing colors since pixal art seems to use high-contrast colors often. This made training more difficult in fact. I believe this is because the model lacks experience understanding "what colors are similar" and vice versa.

Scaling does not always apply! In UNet diffusion, it is tempting to scale when things go wrong. In most cases this will do nothing except make your training slower. For small networks, a better architecture weighs much more than enlargening the network.

Penalties hurt the best players. Penalty functions are good at raising structure from average, but they also seem to hamper "the perfect players". Enforcing saturation rules force already well-trained samples to deviate, losing performance. To alleviate this, we can degrade the penalty over time or, for "obviously easy" tasks, trust the model and delete the penalties altogether.

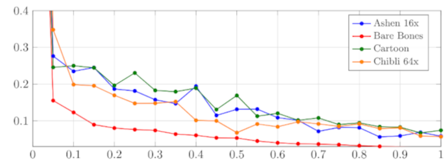
Generalization is costly. There is no magic in the world, any hint of generalization demands larger models. The same model size which handles translating one pack well fails miserably if input includes more than one pack. Without a pre-trained base, the best way to get good results is to keep the data "clean and focused".

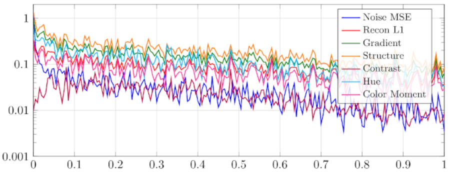
3. Architecture



Overview of the architecture: In Minecraft, the frequency of similar textures is vastly uneven in distribution. To handle large quantities of one-shot or even zero-shot precdictions, we adopted a **dual U-Net diffusion framework**, in which the StyleAwareNet uses cross-attention to better integrate the styling of reference images, and a fallback recoloring that enforces the vanilla texture structure, and recolors the pixels according to the average pixel status of other textures. Based on KNN neighbourhood crowdiness, the network automatically decides which to lean on more and how much penalty is used to bring out structure and balance coloring. The results of the two models are mixed via high/low pass in the selector (or, if one excels, that one is chosen).

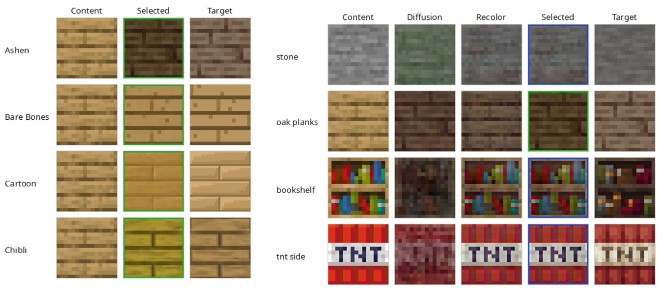
4. Training Process





Training is conducted without overfitting. For diffusion models, the drop of the loss curve is overall smooth, and the decomposition of the loss is rather even. NoiseMSE used to dominate the penalty, but drops fast. Structure is extremely hard for diffusion to keep, so it is the most prominent at last. However, an unexpected high-ranging loss is hue difference.

5. Training Results & Future Work



Overall, **diffusion excels when the reference is strong**, especially for repeating patterns like logs, wood planks and even ores, the last having only one valid reference (the deepslate variation). As can be seen on the left, wood planks are nearly perfectly predicted. However, diffusion suffers when reference data is "off", for instance the greenish tint on stone may be related to multiple mossy variants being used as reference, and **it fails greatly when the texture is very complex**, like TNT. In these cases, the selector **correctly falls back to RecolorNet**, which preserves the structure of vanilla textures but learns the "pallete" of the resource pack. In the future, more work needs be done on training sprite texture, which includes alpha channels, as well as expriments on generalizing the model through better architecture.